

TALL TALE STUDIO

Phase 1 - Complete Codex Implementation Prompt

Authoring Studio | Asset Library | Publishing | Player Runtime | Captain Control | Future Vision
Integration

One shared story model. One authoritative progression engine. Three interfaces: Studio, Player, and Captain. A fourth interface, the future vision helper, must plug into the same verification pipeline without redesigning the platform.

Prepared as a direct, copyable implementation brief for Codex

July 2026

HOW TO USE THIS DOCUMENT

Copy the implementation prompt beginning on the next page into Codex from the root of the intended repository.

The prompt deliberately instructs Codex to inspect the existing application before changing it, preserve unrelated work, adapt route and technology details to the real repository, and continue beyond planning into tested implementation.

This brief treats the future vision helper as an extension of the same verification system used by Captain approval and player confirmation. That is the structural safeguard that prevents Phase 2 from requiring a rewrite.

The prompt is intentionally demanding. Software tends to obey the least convenient interpretation unless one removes its hiding places.

IMPLEMENTATION PROMPT BEGINS

TALL TALE STUDIO - PHASE 1

SUPER-DETAILED CODEX IMPLEMENTATION PROMPT

MISSION

Implement a complete, production-quality Phase 1 Tall Tale Studio inside the existing Tall Tale Companion project. This is not a mockup, isolated demo, or disconnected editor. It must become the authoring system that creates the same Tall Tales the player experience displays and the Captain experience controls.

The implementation must preserve the current player and Captain experiences, integrate with the existing application architecture and visual language, and establish clean extension points for a future local helper application that will perform screen capture and machine-vision verification. Do not implement the helper application or real computer vision in this phase. Do implement the contracts, event model, verification abstractions, identifiers, persistence, and development simulator needed so that vision can be added later without replacing the story engine.

The final system must make it easy to

1. Create a new Tall Tale from a dedicated Studio route.
2. Organize the Tall Tale into chapters.
3. Build the main story by dragging, dropping, configuring, and reordering reusable story blocks.
4. Upload and reuse images and other media through a proper asset library.
5. Pair original and modified images for cinematic transformations.
6. Create locations, artifacts, riddles, notes, directions, confirmations, reveals, and chapter endings.
7. Preview an individual block or play the tale from any point without corrupting real progress.
8. Publish immutable versions.
9. Automatically expose published Tall Tales on the player-facing Tall Tale selection page.
10. Let the Captain monitor and manually control active sessions now.
11. Let future verification providers, including a local machine-vision helper, submit the same standardized completion events later.
12. Extend the platform with additional block types, verification methods, assets, and branching logic without redesigning the database or player runtime.

Treat this as a foundational product architecture task. Implement it carefully, coherently, and end to end.

1. FIRST ACTIONS: INSPECT BEFORE MODIFYING

Before changing code

- Inspect the repository structure, framework, package manager, routing, state management, API layer, persistence layer, authentication, Captain tools, player pages, animation system, media handling, testing conventions, deployment assumptions, and coding standards.
- Identify the canonical source of truth for current Tall Tale content and player progress.

- Identify whether the current project already has concepts similar to tales, chapters, clues, stages, sessions, unlocks, users, Captain controls, assets, or published content.
- Identify current route patterns and preserve them.
- Identify how the player and Captain clients receive live updates, if live updates already exist.
- Identify how database migrations are created and tested.
- Identify how user-facing timestamps, save states, loading states, and errors are displayed.
- Identify the current visual design tokens, spacing, typography, parchment treatments, animation utilities, modal patterns, form controls, and responsive breakpoints.

Write a brief internal implementation map before coding, but do not stop at planning. Continue through implementation, migration, integration, tests, validation, and documentation.

Do not build a parallel mini-application with separate fake data. Use the existing application and database conventions unless a well-justified migration is required.

2. NON-NEGOTIABLE PRODUCT RULES

2.1 End-to-end truth

Every important Studio action must be backed by real persistence and real runtime behavior. Do not ship decorative controls that do nothing.

Examples

- Creating a Tall Tale must create a persistent Tall Tale record.
- Publishing must create a persistent immutable published version.
- A newly published Tall Tale must appear on the player-facing selection page without hand-editing source files.
- Reordering blocks must change the order used by the player runtime.
- Uploading an image must create a reusable asset record and actual stored file.
- Replacing an asset must update the rendered result without breaking every block reference.
- Captain unlock actions must update the active player session through the real progression engine.
- Preview mode must be isolated from real player progress.

2.2 Preserve existing behavior

Do not remove or regress current player features, Captain features, authentication, styling, transitions, story content, routes, or deployment behavior. Migrate existing hard-coded or older tale content safely where required.

2.3 No fake status indicators

Status labels must tell the truth. Never display "Saved," "Published," "Updated just now," "Connected," "Vision ready," or similar claims unless the underlying state confirms it.

Use explicit states such as

- Unsaved changes
- Saving
- Saved at [real timestamp]
- Save failed
- Draft validation failed
- Publishing
- Published as version X
- Captain connection lost
- Awaiting verification
- Manually verified by Captain
- Player-confirmed
- Vision provider unavailable

2.4 Durable identifiers

Use stable unique IDs for tales, draft revisions, published versions, chapters, blocks, assets, asset variants, collections, locations, artifacts, sessions, progress records, unlock events, verification requests, and verification events.

Never use visible names or array indexes as durable foreign keys.

2.5 Extensibility over hard-coding

Block rendering, block configuration, verification handling, and completion logic must be registry-driven or otherwise modular. Adding a new block type later should not require a giant chain of conditionals spread across the entire application.

2.6 Accessibility and responsiveness

The Studio is desktop-first because drag-and-drop authoring benefits from space, but it must remain usable on smaller screens. The player experience must remain fully responsive for desktop and iPhone. Preserve keyboard access, visible focus states, semantic labels, contrast, reduced-motion support, and screen-reader-friendly status messaging.

2.7 Visual quality

Do not produce a generic gray SaaS dashboard. The Studio should feel like a refined creative tool within the existing pirate story world: parchment, cartography, ink, brass, wood, subtle depth, and cinematic motion, while keeping controls readable and efficient.

Authoring interactions should be responsive and not needlessly slow. Player-facing reveals may be longer and theatrical. Avoid meaningless 250 ms fades for major story moments, but also avoid ten-second blank fades that merely waste the player's remaining lifespan.

3. REQUIRED ROUTE AND EXPERIENCE MODEL

Adapt exact route syntax to the existing router, but provide clear separation among the three experiences.

3.1 Player experience

Provide or integrate routes equivalent to

- /tales or the existing player home/catalog route
- /play/:taleSlug
- /play/:taleSlug/session/:sessionId when sessions are explicit
- /play/:taleSlug/history or the existing completed-tale history pattern

The player must see only published, available Tall Tales. Drafts must never appear publicly.

3.2 Captain experience

Provide or integrate routes equivalent to

- /captain
- /captain/tales/:taleId
- /captain/sessions/:sessionId

The Captain must be able to choose a Tall Tale, view active sessions, inspect the current story position, see pending verification requests, manually complete or reject the current verification, trigger permitted presentation events, and recover from mistakes through controlled rollback or jump actions.

3.3 Studio experience

Create a dedicated authoring route equivalent to

- /studio
- /studio/tales/new
- /studio/tales/:taleId
- /studio/tales/:taleId/settings
- /studio/tales/:taleId/assets
- /studio/tales/:taleId/locations
- /studio/tales/:taleId/artifacts
- /studio/tales/:taleId/versions

Exact nested routing may differ, but the Studio must be independently addressable and must not be hidden inside the Captain session-control page.

3.4 Authorization

Use the project's existing authentication and authorization model. At minimum

- Players may view and play eligible published tales.
- Captains may control sessions they are authorized to control.

- Creators/editors may create and modify drafts.
- Only authorized creators/publishers may publish.
- Draft assets, unpublished content, private Captain notes, reference images, internal verification metadata, and Studio APIs must not leak into public player payloads.

If the project currently has a single trusted creator account, still implement permissions through explicit roles/capabilities rather than hard-coding one email address throughout the codebase.

4. CORE DOMAIN MODEL

Use the existing persistence technology and naming conventions, but implement equivalent concepts and relationships.

4.1 Tall Tale

Represents the long-lived identity of an adventure.

Suggested fields

- id
- slug
- title
- subtitle
- shortDescription
- longDescription
- coverAssetId
- themeId or theme configuration
- status: draft, published, archived
- visibility: private, unlisted, public, or the closest supported model
- creatorId
- createdAt
- updatedAt
- currentDraftRevisionId
- latestPublishedVersionId
- defaultPlayerCountMin
- defaultPlayerCountMax
- estimatedDuration
- contentWarnings or notes if supported
- sortOrder or featured state for player catalog ordering

4.2 Draft revision

The editable working state of a Tall Tale. The implementation may use one mutable draft or revisioned drafts, depending on the current architecture, but publishing must create a stable snapshot.

Suggested fields

- id
- taleId
- revisionNumber
- basedOnPublishedVersionId
- createdBy
- createdAt
- updatedAt
- autosaveVersion or optimistic concurrency token
- validationState

4.3 Published version

An immutable snapshot of all player-runtime content required to play a specific version.

Suggested fields

- id
- taleId
- versionLabel, such as 1.0, 1.1, or sequential integer
- versionNumber
- publishedAt
- publishedBy
- releaseNotes
- contentSnapshot or normalized version-bound records
- schemaVersion
- checksum or integrity marker
- isCurrent

A running player session must remain pinned to the published version it started with unless an explicit migration workflow is implemented. Editing or republishing a draft must not silently change an active or completed session.

4.4 Chapter

Suggested fields

- id
- draftRevisionId or publishedVersionId
- title
- subtitle
- description
- orderIndex
- coverAssetId
- estimatedDuration
- entryBlockId
- completionBlockId

- isOptional
- metadata

4.5 Story block

Every unit in the story timeline is a block.

Suggested shared fields

- id
- chapterId
- blockType
- title
- internalLabel
- orderIndex
- configuration JSON or typed child configuration
- presentation configuration
- completion configuration
- nextBlockId for linear flow
- createdAt
- updatedAt
- isEnabled
- creatorNotes, private

The database may normalize block-specific fields or store versioned JSON. Choose the approach that best fits the project, but validate configuration with a schema per block type.

4.6 Block connection

Even though Phase 1 presents a mostly linear vertical timeline, the data model must allow future branching.

Represent connections explicitly or reserve compatible fields for

- sourceBlockId
- targetBlockId
- connectionType: default, choice, hint, success, failure, optional, return
- conditionExpression or future condition reference
- label
- orderIndex

For Phase 1, the editor may maintain one default next connection per block, plus optional connections where required by Choice or Hint behavior.

4.7 Asset

Represents a stable reusable logical asset, not merely a filename.

Suggested fields

- id
- taleId or library scope
- mediaType: image, video, audio, document, model, other
- displayName
- description
- originalFilename
- storageKey
- mimeType
- fileSize
- width
- height
- duration
- checksum
- createdBy
- createdAt
- updatedAt
- currentVariantId or currentFileRevisionId
- deletedAt for safe soft deletion

4.8 Asset file revision or variant

Support replacing or reprocessing a file without changing the logical asset ID.

Suggested fields

- id
- assetId
- role: original, optimized, thumbnail, preview, blurredPlaceholder, mobile, poster, waveform, other
- storageKey
- mimeType
- fileSize
- width
- height
- duration
- checksum
- processingState
- createdAt

Preserve the uploaded original. Generate web-friendly derivatives where the existing stack supports it.

4.9 Asset role assignment

Allow an asset to have multiple semantic roles without duplicating the file.

Roles should include at least

- storyArtwork
- mapImage
- transformationBefore
- transformationAfter
- cinematic
- audioCue
- locationReference
- objectReference
- negativeReference
- cover
- artifactArtwork
- captainOnlyReference

Some roles are future-facing and need not change player behavior yet, but they must be representable.

4.10 Asset collection

Group related media into reusable collections.

Suggested fields

- id
- taleId
- name
- description
- collectionType: general, transformationSet, locationReferenceSet, objectReferenceSet, chapterSet
- locationId optional
- artifactId optional
- createdAt
- updatedAt

Collection membership should support labels such as frontAngle, leftAngle, night, negativeNearbyRock, transformationBefore, transformationAfter, and creator-defined tags.

4.11 Tagging

Support reusable tags and search by tags. Tags should not be stored only as comma-separated display strings if the current database can reasonably normalize them.

4.12 Location

Represents a reusable story location.

Suggested fields

- id
- taleId
- name
- slug or internal code
- region
- generalDescription
- playerFacingDescription
- captainNotes
- mapAssetId
- displayAssetId
- referenceCollectionId
- verificationProfileId optional
- orderIndex
- createdAt
- updatedAt

4.13 Artifact

Represents the story-world meaning of an ordinary in-game object.

Suggested fields

- id
- taleId
- name
- shortDescription
- loreDescription
- ordinaryGameObjectLabel
- artworkAssetId
- revealVideoAssetId
- optionalModelAssetId
- inventoryCategory
- collectionGroup
- sortOrder
- persistentAfterUnlock
- createdAt
- updatedAt

4.14 Player session

Suggested fields

- id
- taleId
- publishedVersionId

- ownerId or partyId
- captainId optional
- status: notStarted, active, paused, completed, abandoned
- currentChapterId
- currentBlockId
- startedAt
- updatedAt
- completedAt
- lastHeartbeatAt if applicable
- previewMode false for real sessions

4.15 Progress and event history

Do not store only one mutable currentBlock value. Preserve an append-only or auditable event history sufficient to explain how progression occurred.

Events should include

- sessionStarted
- blockEntered
- verificationRequested
- verificationSatisfied
- verificationRejected
- captainManualUnlock
- playerConfirmation
- textAnswerAccepted
- blockCompleted
- presentationTriggered
- chapterCompleted
- artifactGranted
- sessionPaused
- sessionResumed
- sessionCompleted
- captainRollback
- captainJump

Every event should include

- id
- sessionId
- publishedVersionId
- blockId where applicable
- eventType
- sourceType
- sourceId or actorId where applicable
- idempotencyKey
- payload
- createdAt

- correlationId or verificationRequestId where applicable

5. FUTURE VISION-HELPER COMPATIBILITY

This section is mandatory. Phase 1 must be designed so that a future local helper application can be added without rebuilding progression.

5.1 Verification provider abstraction

Implement a server-side verification abstraction with provider types equivalent to:

- captainManual
- playerConfirmation
- textAnswer
- timer
- visionLocation
- visionObject
- externalWebhook or futureExternalProvider

Do not implement actual vision inference now. Do implement the shared interface and persisted configuration.

Conceptual interface

```
VerificationProvider.canHandle(type)
VerificationProvider.createRequest(session, block, configuration)
VerificationProvider.evaluateSubmission(request, submission)
VerificationProvider.cancelRequest(request)
```

The progression engine must consume standardized outcomes rather than contain provider-specific logic.

5.2 Verification request model

Suggested fields

- id
- sessionId
- blockId
- providerType
- status: pending, satisfied, rejected, expired, cancelled
- requestedAt
- resolvedAt
- expiresAt optional
- configurationSnapshot
- providerCorrelationId optional
- satisfiedByEventId optional

5.3 Standardized verification event contract

Create an internal and externally documentable event contract similar to

```
{
  "schemaVersion": 1,
  "eventId": "uuid",
  "idempotencyKey": "stable-unique-key",
  "eventType": "verification.observation" or "verification.result",
  "providerType": "visionLocation",
  "providerInstanceId": "local-helper-device-id",
  "sessionId": "uuid",
  "publishedVersionId": "uuid",
  "blockId": "uuid",
  "verificationRequestId": "uuid",
  "observedAt": "ISO-8601 timestamp",
  "result": "match" | "notMatch" | "uncertain",
  "confidence": 0.972,
  "evidence": {
    "referenceCollectionId": "uuid",
    "matchedReferenceAssetId": "uuid",
    "consecutivePassingFrames": 4,
    "windowFrames": 6
  }
}
```

The exact JSON may follow existing API conventions, but include schema versioning, idempotency, session identity, block identity, published-version identity, request identity, provider identity, timestamp, result, confidence, and extensible evidence.

5.4 Security for future helper app

Design but do not fully expose an unsafe public endpoint. Establish a documented plan and server-side model for:

- Pairing a helper instance to a user/session through a short-lived pairing code or device token.
- Scoping the helper to one player session.
- Rotating and revoking tokens.
- Rejecting events for a different session, version, block, or expired verification request.
- Validating idempotency and replay attacks.
- Rate limiting.
- Never accepting arbitrary "advance story" commands from the helper.
- Accepting only verification evidence against an existing pending request.

5.5 Development simulator

Add a developer-only verification simulator in the Captain or Studio preview tooling. It should let an authorized developer submit simulated outcomes for a pending request:

- Match
- Not match
- Uncertain
- Custom confidence
- Repeated duplicate event to verify idempotency
- Stale request event
- Wrong-block event

This simulator proves that future vision integration can use the same event path. It must be clearly labeled as a development/testing tool and unavailable to ordinary players in production.

5.6 Block configuration compatibility

Any block that may later use vision must store provider-independent completion configuration plus provider-specific settings.

Example

```
{
  "completionMode": "verification",
  "verification": {
    "providerType": "captainManual",
    "futureProviderOptions": {
      "visionLocation": {
        "referenceCollectionId": "uuid",
        "coarseThreshold": 0.86,
        "minimumGeometricInliers": 35,
        "requiredPassingFrames": 4,
        "windowFrames": 6
      }
    }
  }
}
```

Do not make manual Captain approval a special one-off field that later has to be deleted. It should be one provider configuration selected from the same verification model.

6. STUDIO INFORMATION ARCHITECTURE

6.1 Studio landing page

Create a polished Studio landing page containing

- Create New Tall Tale
- Continue editing recent drafts
- Published tales
- Archived tales
- Search
- Sort
- Duplicate tale
- Archive tale
- Open player preview
- Open Captain controls
- Last real save timestamp
- Draft validation state
- Latest published version

Each tale card should clearly distinguish Draft, Published, Draft with unpublished changes, and Archived.

6.2 New Tall Tale flow

Create a guided but quick creation flow with

- Title
- Slug generated but editable with validation
- Subtitle
- Short description
- Long description
- Cover image selection or upload
- Theme selection or inherited project default
- Player count range
- Estimated duration
- Visibility
- Initial chapter title

After creation, open the editor immediately.

6.3 Editor shell

The editor should include

Top bar

- Tall Tale title and draft indicator
- Back to Studio
- Undo
- Redo

- Real autosave state
- Preview
- Play From Here when a block is selected
- Validate
- Publish
- More menu for settings, duplicate, archive, version history

Left panel

- Block Library tab
- Chapters tab
- Story Outline tab
- Search blocks
- Collapsible categories: Story, Direction and Location, Media, Reveal, Interaction, Logic

Center canvas

- Vertical chapter timeline
- Chapter headers
- Block cards in order
- Drag handles
- Insertion indicators
- Selected-block state
- Multi-select only if safely implemented
- Empty chapter drop zone
- Add block button between cards
- Clear connection/flow indicators
- Collapsible chapters
- Optional compact overview mode

Right inspector

- Selected block title and type
- Block content fields
- Presentation settings
- Completion/verification settings
- Asset pickers
- Private creator notes
- Validation errors and warnings
- Delete block
- Duplicate block
- Move to chapter

Bottom or slide-up asset drawer

- Images
- Video
- Audio

- Documents
- Collections
- Search
- Filters
- Upload drop zone
- Recently used
- Unused assets
- Drag asset onto compatible block fields

Use the layout pattern that best fits the existing UI framework, but preserve these functional regions.

6.4 Drag-and-drop behavior

Use an established accessible drag-and-drop library already present in the project or add a well-supported one appropriate to the framework.

Required behavior

- Drag a block type from the library into a chapter.
- Drag existing blocks within a chapter.
- Drag blocks across chapters.
- Show exact insertion position.
- Persist order.
- Update default next connections safely.
- Support keyboard reordering or equivalent accessible controls.
- Prevent accidental loss if persistence fails.
- Roll back the local visual order or clearly show unsaved failure if the server rejects the change.
- Do not trigger drag when the user is selecting text or using controls inside a block.

6.5 Undo and redo

Implement practical undo/redo for editor operations if compatible with the current architecture. At minimum support recent local structural and field changes before autosave commits. Do not fake undo buttons.

6.6 Autosave and concurrency

Implement debounced autosave with truthful status. Prevent one stale browser tab from silently overwriting newer changes.

Use optimistic concurrency through a revision number, updatedAt token, ETag, or equivalent.

When a conflict occurs

- Stop claiming the draft is saved.
- Preserve the user's unsaved local work.
- Explain that the draft changed elsewhere.
- Offer a safe reload/compare path consistent with project capabilities.

7. REQUIRED BLOCK REGISTRY AND BLOCK TYPES

Implement a block registry containing, for each block type

- Stable type identifier
- Display name
- Category
- Icon
- Description
- Default configuration
- Validation schema
- Editor inspector component
- Player renderer component
- Preview renderer
- Completion behavior
- Asset field compatibility
- Migration/schema version

Do not scatter block-type knowledge throughout unrelated components.

Implement the following Phase 1 block types.

7.1 Narrative block

Purpose: story narration or exposition.

Fields

- Heading
- Body rich text or safe formatted text
- Optional narrator label
- Optional background asset
- Text alignment
- Width style
- Entrance animation
- Continue behavior: automatic after animation, player button, Captain approval, or configured verification

Player rendering

- Cinematic but readable
- Preserves paragraph spacing
- Supports reduced motion

7.2 Captain's Note block

Purpose: handwritten letter, journal page, or Captain message.

Fields

- Title
- Body
- Signature
- Paper style
- Ink style
- Optional portrait/seal asset
- Optional narration audio
- Page-turn or ink-reveal animation
- Continue button label
- Completion mode

7.3 Riddle block

Fields

- Riddle title
- Riddle text
- Optional illustration
- Optional answer configuration
- Accepted answer variants
- Case sensitivity
- Whitespace normalization
- Optional fuzzy matching threshold only if safely supported
- Attempt limit optional
- Hint sequence
- Hint delay or Captain-controlled hint release
- Wrong-answer feedback
- Completion mode

Do not expose correct answers to public player payloads before submission. Validate answers server-side.

7.4 Information block

Purpose: straightforward rules, context, safety notes, or instructions.

Fields

- Heading
- Body
- Importance level

- Icon
- Optional asset
- Acknowledgment required
- Button label

7.5 Travel Direction block

Fields

- Heading
- Direction text
- Destination visibility
- Compass heading optional
- Region optional
- Estimated travel time optional
- Location reference
- Map asset
- Additional Captain-only notes
- Continue/completion mode

7.6 Location block

Purpose: introduce or represent a reusable destination.

Fields

- Location selection
- Player-facing title override
- Player-facing description override
- Display asset
- Map asset
- Arrival instructions
- Completion mode
- Future vision reference collection
- Future vision profile settings

7.7 Arrival Check block

Purpose: pause progression until the player is considered at the correct place.

Phase 1 provider choices

- Captain manual approval
- Player confirmation button
- Text phrase/code

Future provider choices visible as disabled or "coming later" only if that matches the product design:

- Vision location

Configuration

- Prompt shown to player
- Pending-state text
- Captain notification text
- Verification provider
- Timeout optional
- Allow Captain override
- Future reference collection

7.8 Image block

Fields

- Asset
- Caption
- Alt text
- Display mode: inline, full-bleed, fullscreen, journal frame, map fragment, memory, background
- Object fit/crop/focal point
- Entrance motion
- Optional pan/zoom
- Duration
- Completion mode

7.9 Image Transformation block

This block is central to Phase 1.

Fields

- Before asset
- After asset
- Optional intermediate assets or reserved future support
- Transition preset
- Duration
- Hold-before duration
- Hold-after duration
- Audio cue
- Caption or message revealed
- Alignment data for before and after assets
- Mobile crop/focal point
- Completion mode

Required transition presets should include at least

- Crossfade
- Ink spread
- Ancient carving reveal
- Moonlight reveal
- Fog reveal
- Water wash
- Magical glow
- Camera push-in

Implement the presets with the project's animation system. A preset may initially be composed from CSS, canvas, masks, gradients, image overlays, and existing animation libraries. Do not require creators to write code.

Alignment editor

- Show before and after images overlaid.
- Provide opacity comparison slider.
- Allow translate X/Y, scale, rotation, crop, and focal point adjustments for the after image relative to the before image.
- Provide reset alignment.
- Preview the full transition.
- Persist alignment parameters.

7.10 Cinematic block

Fields

- Video asset
- Poster asset
- Audio behavior
- Autoplay where permitted
- Captions/subtitles asset optional
- Skippable configuration
- Minimum watch duration
- Completion after playback or manual action
- Fallback for browsers that block autoplay

7.11 Audio block

Fields

- Audio asset
- Title
- Optional transcript
- Playback mode

- Loop
- Volume default
- Require completion or allow background continuation
- Fade in/out

Audio may also be attached to other compatible blocks. Avoid double playback or orphaned audio when navigation changes.

7.12 Artifact Reveal block

Fields

- Artifact selection
- Ordinary in-game object label
- Reveal artwork
- Reveal video optional
- Lore title
- Lore description
- Reveal animation
- Audio cue
- Add to collection after reveal
- Completion mode

7.13 Hidden Message Reveal block

Fields

- Base image
- Revealed image or message layer
- Message text optional
- Reveal style
- Duration
- Audio
- Completion mode

This may share rendering infrastructure with Image Transformation but remains a distinct semantic block for creator clarity.

7.14 Collection Update block

Fields

- Artifact or collection item
- Quantity
- Progress label
- Total expected count optional
- Celebration style
- Persist to session inventory

- Optional continue action

The operation must be idempotent. Re-entering or refreshing the block must not grant duplicates unless explicitly configured.

7.15 Confirmation block

Fields

- Prompt
- Primary button label
- Optional secondary action
- Confirmation style
- Captain override

7.16 Choice block

Fields

- Prompt
- Two or more choices
- Choice label
- Optional description and image
- Target block for each choice
- Whether choice is reversible
- Captain visibility

The Phase 1 canvas may show choices inside the inspector and outline even if a full node graph is deferred.

7.17 Text Answer block

Fields

- Prompt
- Accepted answers
- Normalization rules
- Attempt behavior
- Feedback
- Hint integration
- Completion on accepted answer

Validate server-side and avoid exposing accepted answers to the player client.

7.18 Captain Approval block

Purpose: intentional pause until the Captain advances it.

Fields

- Player waiting text
- Captain instruction
- Optional presentation trigger before approval
- Allow reject/request retry
- Timeout behavior optional

Implement it through the common verification request model using provider type captainManual.

7.19 Wait block

Fields

- Duration or wait condition
- Player-facing waiting presentation
- Allow Captain skip
- Continue automatically after condition

7.20 Condition block

Provide a constrained, safe Phase 1 condition builder rather than arbitrary code execution.

Support comparisons against known session variables such as

- Artifact owned
- Hint requested
- Choice selected
- Chapter completed
- Numeric counter
- Boolean flag

Route to configured target blocks. Validate against loops or broken targets where feasible.

7.21 Set Variable block

Support typed variables

- Boolean
- Number
- String from a constrained value

Operations

- Set
- Increment/decrement number
- Toggle boolean

Do not allow arbitrary JavaScript or server code.

7.22 Chapter Complete block

Fields

- Completion message
- Summary
- Reward/artifact optional
- Next chapter behavior
- Return-to-map behavior
- Animation

7.23 Tale Complete block

Fields

- Finale content
- Completion message
- Credits or dedication
- Final collection state
- Replay availability
- History entry presentation

8. ASSET LIBRARY IMPLEMENTATION

8.1 Upload experience

Support

- Drag-and-drop upload
- Multi-file selection
- Paste image from clipboard where browser support allows
- Upload progress per file
- Validation before upload
- Clear errors
- Retry

- Cancellation where practical
- Duplicate detection using checksum
- Friendly handling of duplicate names

Do not require creators to manage server paths or filenames.

8.2 Supported Phase 1 media

At minimum

- PNG
- JPEG
- WebP
- AVIF if existing tooling supports it reliably
- GIF only if the product intentionally supports animation
- MP4/WebM for video according to browser and server support
- MP3/Ogg/WAV according to existing stack
- PDF or document assets only if the player already supports them; otherwise store them as creator resources without pretending they render in-player

Enforce configured size limits and MIME validation server-side, not only by filename extension.

8.3 Image processing

Preserve the original upload. Generate derivatives appropriate to the existing backend:

- Thumbnail
- Medium preview
- Optimized player version
- Optional blurred placeholder
- Optional mobile derivative

Store dimensions and metadata. Strip dangerous metadata where appropriate while preserving required orientation.

8.4 Storage architecture

Use the project's existing object storage or file-storage abstraction. If none exists, create a storage service interface that supports local development storage now and future object storage later.

Do not store large raw binary media directly inside ordinary relational table rows unless the existing architecture explicitly requires it.

Use safe opaque storage keys, not untrusted user filenames as paths.

8.5 Asset browser

Required filters

- Media type
- Chapter
- Location
- Artifact
- Role
- Tag
- Collection
- Recently used
- Unused

Required actions

- Select
- Multi-select where useful
- Edit metadata
- Assign tags
- Assign roles
- Add to collection
- Replace file
- Download original for authorized creator
- Archive/soft-delete
- View usages

8.6 Usage tracking and safe deletion

Before deletion, show every block, location, artifact, cover, or collection that uses the asset.

Do not allow destructive deletion of a referenced asset without an explicit replacement or safe unlink process.

8.7 Replace asset behavior

Replacing the file behind a logical asset must preserve the asset ID and all references. Keep prior file revisions for rollback or audit if practical.

8.8 Asset picker

Create a reusable asset picker component for inspector fields. It should support:

- Search
- Filters
- Preview

- Upload new
- Select existing
- Clear selection
- Required-role hints
- Compatibility checks by media type

8.9 Asset collections

Allow creators to create collections from the Asset Library and from location/artifact editors.

Example collection layout

Sailor's Bounty Message Rock

Presentation

- Normal rock
- Carved rock
- Close-up message
- Night reveal video

Future recognition references

- Front angle
- Left angle
- Right angle
- Distance view
- Lantern view

Negative references

- Nearby wrong rock
- Southern cave wall
- Similar shoreline

Phase 1 does not analyze these references, but it must preserve their roles and collection relationships.

9. LOCATION AND ARTIFACT LIBRARIES

9.1 Locations page

Provide a location management view with

- Create location
- Search
- Sort

- Display image
- Map image
- Region
- Player description
- Captain notes
- Reference collection
- Used-by blocks
- Duplicate
- Archive

9.2 Location editor

Include tabs or sections for

- Overview
- Player presentation
- Maps and imagery
- Verification references
- Usage
- Captain notes

The verification references section should explain that images can be collected now for a future vision helper. It should support assigning `locationReference` and `negativeReference` roles without claiming that recognition is active.

9.3 Artifacts page

Provide an artifact library with

- Artwork
- Name
- Ordinary in-game object
- Collection group
- Reveal media
- Lore
- Used-by blocks
- Archive

9.4 Artifact persistence in sessions

When a Collection Update or Artifact Reveal grants an artifact, store it against the session or player inventory according to existing product behavior. Ensure idempotency and correct history display.

10. PREVIEW AND TESTING WORKFLOWS

10.1 Block preview

From any selected block, allow the creator to render only that block using its draft configuration and assets.

Requirements

- Isolated preview state
- No real session mutation
- Accurate desktop and mobile viewport preview
- Reduced-motion preview option
- Replay transition
- View validation errors

10.2 Play From Here

Create a temporary preview session pinned to the current draft revision.

Requirements

- Start from selected block
- Optionally seed required prior variables/artifacts
- Clearly label Preview Mode
- Never appear as a real active player session
- Never write to real player history, achievements, or completed-tale records
- Allow Captain simulation in a separate preview control panel or embedded test drawer
- Dispose or expire preview sessions

10.3 Full Tale preview

Allow a creator to play the entire draft without publishing, still isolated from real progress.

10.4 Device preview

At minimum, provide desktop and common mobile viewport presets. Do not pretend a pixel-perfect emulator is a real device, but make responsive testing convenient.

11. DRAFT VALIDATION AND PUBLISHING

11.1 Validation engine

Before publishing, validate the complete draft.

Errors that block publishing should include

- Missing title or invalid slug
- No chapters
- Chapter without an entry block
- Broken or missing default connections
- Block references a deleted or unavailable asset
- Required field missing
- Choice target missing
- Condition target missing
- Unreachable required block
- Obvious infinite loop where detectable
- Missing Tale Complete endpoint unless intentionally configured otherwise
- Invalid text-answer configuration
- Unsupported media type
- Asset processing failed
- Future vision provider selected as active even though not available in Phase 1
- Private/internal asset assigned to a public player field

Warnings may include

- Unused assets
- Empty optional descriptions
- No cover image
- Very large media
- No mobile crop
- Missing alt text
- Chapter estimated duration missing
- Block cannot be reached except through Captain jump

Display validation issues in a panel with links that focus the affected chapter, block, asset, location, or artifact.

11.2 Publish flow

Publishing must

1. Re-run server-side validation.
2. Create an immutable published version.
3. Snapshot or bind every required chapter, block, connection, public asset reference, location presentation, artifact presentation, condition, and completion configuration.
4. Assign version number/label.
5. Record publisher and timestamp.
6. Mark it as current when requested.
7. Make it eligible for the player catalog according to visibility and availability.
8. Leave the working draft available for future edits.
9. Record release notes.
10. Broadcast catalog invalidation/update through the existing data-refresh mechanism where applicable.

11.3 Unpublished changes

After publishing, any draft edit must visibly change the tale state to "Published with unpublished changes" rather than silently altering the published version.

11.4 Version history

Provide a version history page showing

- Version
- Publish timestamp
- Publisher
- Release notes
- Current/non-current status
- Number of active sessions pinned to version
- Preview published version
- Compare summary if feasible
- Restore/copy published version into a new draft, without mutating the old version

11.5 Catalog eligibility

A published Tall Tale should appear on the player page only when

- A current published version exists.
- Visibility permits the current player.
- It is not archived.
- Availability rules, if any, pass.

Draft-only tales must never appear.

12. PLAYER EXPERIENCE INTEGRATION

12.1 Dynamic tale catalog

Replace or augment any hard-coded tale list with a real catalog query backed by published Tall Tales.

Each player-facing card should support

- Cover art
- Title
- Subtitle or description
- Estimated duration
- Player count

- Status: new, in progress, completed, replayable, unavailable
- Continue button when progress exists
- Start button when eligible
- Replay button when allowed
- Version pinning handled internally

12.2 New tale availability

When an authorized creator publishes a new Tall Tale, it must become visible on the player page through the normal data path. No source-code edit, static JSON edit, manual route addition, or restart-specific hack should be necessary beyond normal deployment/cache invalidation.

12.3 Starting a tale

Starting must

- Create a real player session.
- Pin it to the current published version.
- Enter the first chapter and entry block.
- Record history events.
- Associate or invite a Captain according to current product rules.
- Render the correct block through the shared player block registry.

12.4 Runtime block rendering

The player runtime must render published block configurations, not Studio-only draft objects.

Block transitions should

- Persist progress server-side.
- Be idempotent.
- Resume correctly after refresh.
- Avoid replaying one-time grants.
- Allow replaying presentation where explicitly permitted without re-completing progression.
- Coordinate audio cleanup.
- Respect reduced motion.

12.5 History

Completed and prior sessions should remain viewable using the published version they used. Do not resolve old sessions against whichever draft happens to exist today.

13. CAPTAIN CONTROL INTEGRATION

13.1 Captain dashboard

Show

- Available Tall Tales
- Active sessions
- Waiting-for-Captain sessions
- Player names/party label where available
- Current chapter
- Current block
- Time waiting
- Last real event timestamp
- Connection status if live connections exist
- Preview sessions clearly separated

13.2 Session control page

Show

- Tale and pinned version
- Player/party
- Current chapter and block
- Current player-facing content summary
- Pending verification request
- Recent event timeline
- Session inventory/artifacts
- Variables relevant to debugging
- Captain-only instructions from current block
- Available actions

13.3 Required Captain actions

Implement

- Approve pending Captain verification
- Reject/request retry where supported
- Manually satisfy a player-confirmation or future verification request through explicit override
- Trigger a configured presentation event without duplicating progression
- Pause/resume session
- Roll back to previous safe block
- Jump to a selected block with a warning and explicit reason
- Re-send current block/state to connected player if live synchronization supports it
- Release hint

- Add Captain note to session history if supported

Every destructive or progression-changing action must

- Require authorization.
- Use a confirmation dialog where appropriate.
- Record actor, reason, timestamp, source block, target block, and resulting state.
- Be idempotent where applicable.

13.4 Manual control through common verification

Captain approval must satisfy an existing pending verification request. Do not directly mutate currentBlock from the UI without passing through progression rules.

13.5 Live synchronization

Use the existing real-time mechanism if one exists. If none exists, implement the smallest reliable abstraction consistent with the current stack, such as WebSocket, server-sent events, or short polling.

Required behavior

- Player sees Captain approval without a full manual refresh.
- Captain sees player progression and pending requests promptly.
- Reconnection fetches canonical state.
- Duplicate messages do not duplicate completion.
- UI connection indicators are truthful.

14. STORY RUNTIME AND PROGRESSION ENGINE

14.1 Single authoritative engine

Create or refactor toward one progression service responsible for

- Entering blocks
- Creating verification requests
- Accepting standardized verification outcomes
- Completing blocks
- Executing idempotent side effects
- Applying variables
- Granting artifacts
- Resolving next connections
- Completing chapters
- Completing tales
- Recording events

- Returning the next canonical player state

The Studio preview, real player runtime, Captain controls, and future vision helper must all use this engine or a safely isolated preview instance of it.

14.2 Idempotency

All completion and side-effect operations must tolerate retries.

Examples

- Duplicate Captain approval must not advance twice.
- Browser refresh during Artifact Reveal must not grant the artifact twice.
- Replayed helper event must not complete a later block.
- A late response for a cancelled request must be ignored and recorded safely.

14.3 Transaction boundaries

Where supported, complete a block, apply side effects, create the next block-entry event, update session state, and create the next verification request within a safe transaction or equivalent consistency boundary.

14.4 State reconstruction

The current session state should be explainable from persisted state and history. Do not rely only on transient client state.

14.5 Unknown block types

Published content should be schema-versioned. If a client encounters an unknown block type:

- Do not crash into a blank screen.
- Show a controlled incompatibility message.
- Record diagnostics.
- Prevent unsafe progression.

15. API AND SERVICE BOUNDARIES

Adapt naming to the existing backend. Implement equivalent service boundaries.

Studio APIs/services

- List/create/read/update/archive tales
- Create/update/reorder/delete chapters

- Create/update/reorder/duplicate/delete blocks
- Manage connections
- Upload/list/update/replace/archive assets
- Manage tags and collections
- Manage locations
- Manage artifacts
- Validate draft
- Create preview session
- Publish draft
- List published versions

Player APIs/services

- List eligible published tales
- Read public tale summary
- Start/resume session
- Read canonical current player state
- Submit allowed player interaction
- Submit text answer
- Confirm player action
- Read collection/history

Captain APIs/services

- List controlled/active sessions
- Read session detail
- Approve/reject verification
- Override verification
- Release hint
- Pause/resume
- Rollback/jump
- Trigger permitted presentation

Future helper API/service boundary

- Pair helper
- Read current scoped verification request
- Submit standardized verification observation/result
- Heartbeat/status
- Revoke pairing

Only scaffold the future helper boundary and simulator now. Do not expose insecure unauthenticated advancement endpoints.

16. SECURITY, PRIVACY, AND MEDIA SAFETY

- Enforce authorization server-side.

- Validate all block configuration server-side.
- Sanitize rich text or use a safe structured editor format.
- Prevent path traversal in uploads.
- Validate MIME type and file signatures where practical.
- Store uploads under opaque keys.
- Prevent draft/private media from leaking through public URLs or payloads.
- Use signed URLs or authorized media endpoints if the existing storage model requires access control.
- Rate limit answer submissions, Captain actions, upload endpoints, and future helper events as appropriate.
- Do not log correct riddle answers, sensitive tokens, or full private asset URLs unnecessarily.
- Record security-relevant Captain overrides.
- Keep future vision evidence local by default in the eventual helper design. Phase 1 should not claim to upload or retain screenshots.

17. MIGRATION OF EXISTING CONTENT

If existing Tall Tale content is hard-coded, stored in static files, or represented by a simpler schema:

- Create a migration or import path into the new Tall Tale, chapter, block, asset, and published-version model.
- Preserve the current playable Forever Treasure experience.
- Preserve current URLs or add safe redirects.
- Preserve current active progress where feasible.
- Do not require the user to manually rebuild existing content in the Studio merely to keep the site working.
- Document any content that cannot be migrated automatically.

Create one valid editable draft corresponding to the current tale and one published version that reproduces the current player experience as closely as practical.

18. VISUAL AND INTERACTION REQUIREMENTS

18.1 Studio visual language

The Studio should feel like a premium Cartographer's Table rather than a generic CMS.

Use the existing design system and consider

- Layered parchment surfaces
- Subtle map-grid or chart lines
- Brass/wood accents
- Inked section labels
- Restrained glass only where already used
- Soft elevation and shadows
- Clearly differentiated interactive controls
- Animated but efficient panel transitions

Do not sacrifice readability for theme.

18.2 Motion

Studio motion

- Smooth panel opening
- Clear drag lift and drop settling
- Chapter collapse/expand
- Asset drawer movement
- Save-state transitions
- Preview opening

Player motion

- More cinematic and deliberate
- Transition durations configurable per block
- Major image transformations capable of multi-second sequences
- No blank waiting periods disguised as cinematic pacing

18.3 Empty and error states

Design useful states for

- No Tall Tales
- No chapters
- Empty chapter
- No assets
- Upload failed
- Asset processing
- Draft conflict
- Validation failed
- No published version
- No active Captain sessions
- Waiting for Captain
- Connection lost
- Preview session expired

18.4 Confirmation dialogs

Use explicit confirmation for

- Archive tale
- Delete referenced block
- Delete/replace referenced asset

- Publish
- Rollback player session
- Jump player session
- Reset preview
- Discard unsaved changes

19. PERFORMANCE AND RELIABILITY

- Lazy-load thumbnails and large media.
- Avoid loading original full-resolution images into every asset grid card.
- Paginate or virtualize large asset libraries.
- Debounce search and autosave.
- Cache published catalog safely with explicit invalidation.
- Ensure block editor rerenders remain manageable with long tales.
- Do not upload the same file repeatedly during autosave.
- Resume or safely retry interrupted uploads where practical.
- Use background derivative generation if the current stack supports jobs; otherwise return a truthful processing state.
- Never block the entire editor while one thumbnail processes.

20. TESTING REQUIREMENTS

Add automated tests using the repository's existing test stack.

20.1 Unit tests

Cover

- Block schema validation
- Block registry behavior
- Slug generation/validation
- Draft validation
- Connection resolution
- Text-answer normalization
- Verification-provider dispatch
- Idempotency handling
- Artifact grant idempotency
- Condition evaluation
- Published-version immutability
- Catalog eligibility

20.2 Service/integration tests

Cover

- Create tale -> add chapter -> add blocks -> upload asset -> publish -> player catalog displays tale
- Draft edits after publishing do not alter published session behavior
- Start session pins correct version
- Captain approval satisfies pending request and advances once
- Duplicate approval does not advance twice
- Player confirmation uses same verification outcome path
- Simulated vision match uses same verification outcome path
- Stale simulated vision event is rejected
- Wrong-block simulated event is rejected
- Artifact reveal refresh does not duplicate collection item
- Preview session does not appear in real player history
- Asset replacement preserves block references
- Deleting a referenced asset is blocked or safely handled
- Current existing tale still plays after migration

20.3 End-to-end tests

Where the project supports browser E2E testing, cover the main golden path

1. Creator opens Studio.
2. Creates Tall Tale.
3. Creates first chapter.
4. Drags Narrative, Riddle, Arrival Check, Image Transformation, Artifact Reveal, and Tale Complete blocks into the timeline.
5. Uploads before and after images.
6. Configures alignment and previews transformation.
7. Validates and publishes.
8. Opens player catalog and sees the tale.
9. Starts a session.
10. Player reaches Arrival Check.
11. Captain sees pending request and approves.
12. Player receives transition without manual refresh.
13. Image transformation plays.
14. Artifact is granted once.
15. Tale completes and appears in history.

20.4 Accessibility tests

At minimum

- Keyboard navigation through Studio controls
- Accessible names for icon buttons
- Focus handling in dialogs and drawers
- Drag-and-drop alternative controls
- Alt-text validation for player-facing images
- Reduced-motion behavior

- Status announcements for saving, publishing, errors, and Captain approval

20.5 Migration tests

- Existing data migrates successfully.
- Rollback or backup path is documented.
- Existing published tale remains playable.
- No orphaned current sessions are created.

21. OBSERVABILITY AND DIAGNOSTICS

Add structured diagnostics consistent with the project

- Draft save failures
- Publish validation failures
- Publish completion
- Asset upload/processing failures
- Session progression failures
- Verification request creation and resolution
- Captain overrides
- Duplicate/idempotent event handling
- Unknown block type
- Real-time connection failures

Do not log sensitive answer values or private tokens.

Create a creator-visible diagnostics area or clear error details where appropriate, not merely console errors.

22. DOCUMENTATION

Add or update project documentation covering

- Tall Tale Studio overview
- Domain model
- Block registry and how to add a new block type
- Draft versus published versions
- Asset storage and derivatives
- Player session/version pinning
- Captain control flow
- Verification provider abstraction
- Future local helper pairing/event contract
- Development verification simulator
- Migration of existing content
- Required environment variables

- Storage configuration
- Test commands
- Build/run commands

Include a concise "Future Vision Helper Integration" document with

- Pairing concept
- Event schema
- Security assumptions
- Request lifecycle
- Idempotency rules
- Match/not-match/uncertain semantics
- Why the helper may satisfy only an existing pending verification request
- Which Phase 1 tables/services/endpoints it will use

23. IMPLEMENTATION ORDER

Use the following order unless repository realities require a documented adjustment.

Phase A - Audit and domain foundation

- Inspect current architecture.
- Define migrations and model relationships.
- Add block registry and schemas.
- Add draft/published distinction.
- Add asset storage abstraction.
- Add progression and verification abstractions.

Phase B - Runtime integration

- Migrate or adapt existing tale content.
- Render published blocks in the player experience.
- Implement session pinning and progression events.
- Route Captain approval through common verification.

Phase C - Studio shell

- Studio landing page.
- New tale flow.
- Chapter/timeline editor.
- Inspector.
- Autosave and validation status.

Phase D - Asset/location/artifact tools

- Upload pipeline.
- Asset browser/picker.
- Collections and roles.
- Location library.
- Artifact library.
- Image transformation alignment editor.

Phase E - Preview and publishing

- Block preview.
- Play From Here.
- Draft validation.
- Immutable publishing.
- Version history.
- Dynamic player catalog.

Phase F - Captain and future-provider seam

- Captain dashboard/session control integration.
- Live state updates.
- Development verification simulator.
- Helper event contract and docs.

Phase G - QA and polish

- Automated tests.
- Responsive checks.
- Accessibility checks.
- Migration validation.
- Visual polish.
- Documentation.

Do not call the task complete after producing only Phase A or UI scaffolding. Complete the end-to-end Phase 1 scope.

24. REQUIRED ACCEPTANCE CRITERIA

The implementation is complete only when all of the following are true.

Studio and authoring

- An authorized creator can open a dedicated Studio route.
- A creator can create, rename, duplicate, and archive a Tall Tale.

- A creator can create and reorder chapters.
- A creator can drag new blocks into the story and reorder them.
- Every required Phase 1 block type is registered, editable, validated, previewable, and renderable, or any deliberately deferred secondary block is explicitly documented with no broken placeholder controls.
- Autosave is real and truthful.
- Conflicts do not silently overwrite newer work.

Assets

- A creator can drag-and-drop multiple images.
- Uploaded files persist.
- Thumbnails/optimized variants are generated or truthfully marked as processing.
- Assets are searchable and reusable.
- Assets support tags, roles, collections, and usage tracking.
- Replacing an asset preserves logical references.
- Referenced assets cannot be silently destroyed.
- Before/after transformation images can be aligned and previewed.

Publishing

- Server-side validation prevents broken publication.
- Publishing creates an immutable version.
- Draft changes after publishing do not mutate the published version.
- Version history is visible.
- The newly published tale appears on the player catalog through real data.

Player

- The player can start the new tale.
- The session pins the published version.
- Blocks render from the published block registry.
- Progress survives refresh.
- One-time effects remain idempotent.
- Completion appears in player history.

Captain

- The Captain sees active sessions and current block.
- The Captain sees pending verification.
- Captain approval advances the player through the standard verification flow.
- Reject/retry and override behavior are recorded.
- Rollback and jump are controlled and audited.
- Player state updates without requiring an unreliable manual refresh.

Future vision compatibility

- Verification requests are persisted.
- Provider types include future vision providers.

- Standard event schema is implemented or formally versioned in code.
- Development simulator can submit match, not-match, and uncertain events.
- Simulated match advances only the correct pending request.
- Duplicate, stale, wrong-session, wrong-version, and wrong-block events are rejected or safely ignored.
- No helper can send an unrestricted "advance" command.
- Future reference images and negative references can already be stored in collections.

Quality

- Existing tale behavior is preserved or migrated.
- Tests pass.
- Lint/type checks pass.
- No obvious dead controls remain.
- No fake saved/published/connected claims remain.
- Responsive layouts work at target desktop and mobile sizes.
- Accessibility basics are verified.
- Documentation is complete.

25. DELIVERABLES AND FINAL REPORT

When implementation is finished, provide a precise final report containing

1. Outcome summary.
2. Repository path and branch.
3. Starting and ending commit identifiers if commits are permitted.
4. Every migration added and its purpose.
5. Major architecture decisions.
6. Exact Studio routes.
7. Exact player routes affected.
8. Exact Captain routes affected.
9. Block types implemented.
10. Asset storage behavior.
11. Draft/publish/version behavior.
12. Existing-content migration result.
13. Future vision-helper extension points.
14. Development simulator instructions.
15. Test commands and results.
16. Build/lint/type-check results.
17. Known limitations that are real and specific.
18. Manual validation steps.
19. Files changed, grouped by subsystem.
20. Whether any unrelated pre-existing changes were present and whether they were left untouched.

Do not claim success for tests, migrations, publishing, image processing, live synchronization, or player integration unless actually verified.

26. SOURCE CONTROL AND CHANGE DISCIPLINE

- Work only in the intended repository.
- Inspect git status before changing anything.
- Preserve unrelated pre-existing changes.
- Do not reset, discard, overwrite, or "clean up" unrelated work.
- Keep migrations additive and reversible where practical.
- Use focused commits if repository policy permits commits.
- Do not push, deploy, tag, or merge unless separately authorized.
- Do not alter production secrets or credentials.
- Do not replace current data with sample data.
- Seed/demo content must be clearly separated from real content.

27. FINAL PRODUCT INTENT

This platform should let a creator build a Tall Tale as a sequence of meaningful, reusable story blocks instead of hand-coding every adventure. The Studio, player experience, Captain controls, and future vision helper are not separate products. They are different interfaces over one shared, versioned story model and one authoritative progression engine.

The correct architectural relationship is

Tall Tale Studio

```
-> creates a draft story graph and reusable assets
-> validates and publishes an immutable version
-> automatically appears in the player catalog
-> starts version-pinned player sessions
-> produces pending interaction or verification requests
-> receives player, Captain, timer, or future vision outcomes through one standardized
verification path
-> advances through one authoritative progression engine
-> records an auditable history
-> preserves completed adventures and memories exactly as they were played
```

Implement the full Phase 1 foundation with enough polish that it feels intentional now and enough modularity that automatic visual verification can be attached later without rebuilding the platform.

END OF IMPLEMENTATION PROMPT

REFERENCE ARCHITECTURE AT A GLANCE

Subsystem	Responsibility
Tall Tale Studio	Creates and validates editable draft content, reusable assets, locations, artifacts, blocks, and connections.
Publishing service	Freezes an immutable, schema-versioned release and makes eligible tales available to the player catalog.
Player runtime	Starts version-pinned sessions and renders published blocks through the shared block registry.
Captain controls	Observes sessions and resolves pending verification requests through audited manual actions.
Verification layer	Normalizes Captain, player, timer, text-answer, simulator, and future vision outcomes.
Progression engine	Completes blocks idempotently, applies side effects, records history, and determines the next canonical state.
Future helper app	Pairs to one session and submits evidence only for an existing pending vision request. It cannot issue arbitrary advancement commands.

Critical invariant: No interface advances a session by directly editing the current block. Every outcome passes through the same verification and progression services.